

Unified Smart Reporter

Complete Redesign Plan — Revised with Gap Analysis

February 2026

Core Principles

1. **One entry point** — Smart Reporter is the single radiologist assistant
2. **One generator pipeline** — Admin and user get identical output quality and format
3. **Cache everything** — Every systematic approach is reusable medical knowledge
4. **Brand-consistent** — All UI uses documented CSS variables from STYLE_GUIDE.md
5. **Think like a radiologist** — The approach to reading a scan is standardized; only the findings vary
6. **Resource-aware generation** — AI generators accept user-supplied references (URLs, PDFs, existing DB content) to ground output in evidence — falling back to AI knowledge when none are provided
7. **Flexible entry paths** — Users can enter the editor at any point: via walkthrough, by pasting an existing report, or by generating a blank template

Gap Analysis: Identified Additions

Four gaps were identified after initial planning. Each is integrated into the relevant sections below, and summarised here for traceability.

Gap #	Title	Summary
1	Resource-Aware AI Generation	All generators accept external references (URLs, existing DB cases, TNM data, uploaded PDFs). AI uses them to ground output; falls back to own knowledge when none provided.
2	Skip Algorithm → Paste Report	Users who already have a report can skip the walkthrough entirely, paste their report into the editor, and use AI Assistant panel immediately.
3	Abort Algorithm at Any Stage	Users can exit the walkthrough mid-way through. Options: paste own report, or generate a standard blank template as starting base. Both lead to the same editor.
4	Personal Templates on Paste	When a user pastes a report (via Gap 2 or 3), prompt them to save it as a personal template. Personal templates are separate from admin content and from the shared AI cache.

Part A: Unified Generator Pipeline (Admin = User)

Problem

Currently three separate generators produce different outputs. When admin generates a tool, users get a rich interactive version. When a user asks Smart Reporter about the same topic through the walkthrough, they get a simpler JSON tree.

Solution: One Pipeline, Two Output Tiers

A single module — `clinical_tool_generator.py` — with two generation modes:

- **Mode 1: Quick Walkthrough** (~15–30s) — produces JSON algorithm tree for step-by-step guided reporting. Served immediately. Uses Sonnet, `max_tokens=4000`, `temperature=0.3`.
- **Mode 2: Full Interactive Tool** (~60–120s) — produces complete self-contained HTML with interactive calculator/decision tree. Same output as current admin generators. Uses Sonnet, `max_tokens=20000`, `temperature=0.3`. Generated in background after Mode 1 is served, or pre-generated by admin.

⚠ Gap 1 Integration: Resource-Aware Generation

Both modes accept an optional `resources` parameter. This parameter supports four resource types:

- **Reference URLs** — Article links (e.g. PubMed, Radiopaedia, society guidelines). The generator prompt instructs the AI to incorporate these as evidence sources.
- **Existing DB cases** — Slugs or IDs referencing `IncidentalFindingCalculator`, `ReportingTemplate`, or `ClinicalProtocol` records already in the database. Their content is injected into the prompt context.
- **TNM / scoring data** — Entries from the `view_tnm` tables. If a user's query involves staging (e.g. "lung cancer staging"), the pipeline auto-fetches matching TNM data and feeds it to the generator.
- **Uploaded PDFs** — Admin or user can upload a PDF. The pipeline extracts text (via `PyPDF2` / `pdfplumber`) and injects it as reference context.

If no resources are provided, the AI falls back to its own medical knowledge — identical to current behaviour.

Unified function signature

```
def generate_clinical_tool(topic, mode='walkthrough', context={}, resources=None, model=None)
```

The `resources` parameter is a dict with optional keys: `urls` (list of strings), `db_refs` (list of {type, slug}), `tnm_refs` (list of staging system slugs), `pdf_texts` (list of extracted text strings).

Shared utilities in the module

- `format_resources_for_prompt(resources_json)` — converts all resource types into a unified prompt section
- `validate_quality(html, tool_type)` — shared validation with type-specific checks
- `extract_algorithm_summary(html, title, tool_type)` — shared BeautifulSoup extraction
- `extract_html_content(html)` — shared style/body extraction for app integration

- `extract_pdf_text(file_path)` — NEW: extracts text from uploaded PDF for prompt injection
- `fetch_db_references(db_refs)` — NEW: loads existing DB content by type and slug
- `fetch_tnm_data(tnm_slugs)` — NEW: loads TNM staging data for prompt context

Part B: Smart Caching Strategy

Cache Hierarchy (checked in order)

1. **Admin-verified tools** (highest priority) — IncidentalFindingCalculator / ReportingTemplate with verified_at IS NOT NULL, is_available=True
2. **Admin-generated (unverified)** — same tables, is_available=True but no verified_at
3. **User-generated cache** — ReportingTemplate with category='smart_reporter_cache'. Auto-generated by unified pipeline.

Gap 4 Integration: Personal Templates (New Cache Tier)

A fourth tier is added below the shared cache:

1. **Personal templates** — ReportingTemplate with category='personal_template' and user_id set. These are user-saved reports (from paste or completed walkthrough). Only visible to the owning user. Not shared, not admin-curated.

Personal templates appear in the user's "Your Templates" section on the landing page and can be loaded into the editor as a starting base at any time.

Topic Normalisation

The Haiku intent classifier returns a canonical_topic slug, display_title, modality, body_section, and category. This enables cache lookup by normalised slug across all tiers.

What is NOT cached

- AI Assist responses (corrections, answer, insights) — specific to user's report text
- Quick Check results — specific to user's report text
- Anatomy AI responses (these ARE cached if generated, as protocols with category='anatomy')

Part C: New Smart Reporter UX

Landing Page Redesign

Replace the current 3-card layout + form with a single adaptive input bar, quick-pick chips, and tabbed content sections below.

 Gap 2 Integration: Skip-to-Editor Entry Path

The landing page adds a prominent “I already have a report” action alongside the main input. This allows users who have a completed or draft report to bypass the walkthrough entirely and go straight to the editor with the AI Assistant panel active.

Landing page layout

The top section contains the search input (“What are you working on?”) with quick-pick chips below it. A secondary row offers two explicit paths:

- **Start guided walkthrough** — default path, triggers intent classification and cache lookup
- **Paste my report** — opens the editor directly with an empty report area ready for paste (Gap 2)
- **Browse templates** — opens template/tool browser

Below these, the existing sessions table, templates table, and community reports table remain.

Intent Routing

On submit, the system calls `/api/smart-reporter/route-intent` which returns `intent`, `canonical_topic`, `modality`, `body_section`, etc. Routing:

Intent	Action
walkthrough	Check cache → serve walkthrough OR generate. Open walkthrough scene.
report_help	Open editor directly. Auto-trigger Quick Check for spelling/grammar. Pre-fill AI Assistant for impression/recommendation queries.
tool_request	Open editor with Tool Panel showing matched calculator/scoring tool.
reference	Open editor with Reference Panel showing matched content previews.
protocol	Open editor with Protocol Panel showing matched protocols.
anatomy	Open editor with Anatomy Panel (DB-first, AI fallback).
paste_report	Open editor with empty report area — from “Paste my report” button (Gap 2).

Walkthrough Scene: Abort & Flexible Exit (Gap 3)

⚠ Gap 3 Integration: Abort Algorithm at Any Stage

The walkthrough scene must provide a persistent exit mechanism at every step, not just at completion. Users who realise mid-way that they already know what to report, or who want a blank template, should never feel trapped.

Walkthrough toolbar additions

A persistent toolbar at the top or bottom of the walkthrough scene includes:

- **Exit to Editor** — leaves the walkthrough, opens the editor. If any walkthrough steps were completed, the partial findings are pre-populated into the report draft. The user can then paste their own report text over it or continue editing.
- **Generate Blank Template** — triggers a quick AI call (Haiku, ~5s) to generate a structured but empty reporting template for the current modality/body region. Opens the editor with this template pre-loaded. The user fills in their own findings.
- **Paste My Report** — identical to the landing page path: opens the editor with an empty report area for pasting.

All three paths lead to the same editor interface. The only difference is what content (if any) is pre-loaded in the report area.

State preservation on abort

When a user aborts mid-walkthrough, the system saves the partial walkthrough state (completed steps + answers) to the session. If the user returns to the same session later, they can choose to resume or start fresh.

Personal Template Prompt (Gap 4)

⚠ Gap 4 Integration: Save as Personal Template

When a user pastes a report into the editor (via Gap 2 or Gap 3 exit), the system detects this is externally-sourced text and shows a non-intrusive prompt:

“Would you like to save this as a personal template for future use?” with [Save as Template] and [No thanks] buttons.

If the user saves:

- The report text is stored as a new ReportingTemplate with category='personal_template' and user_id set to the current user.
- The user is prompted for a template name (with a sensible default inferred from content, e.g. “CT Abdomen — Pancreatitis”).
- Personal templates appear in the “Your Templates” tab on the landing page.
- Personal templates can be loaded, edited, renamed, or deleted by the user at any time.
- Personal templates are never shared to other users or to the community — they are private.

This is architecturally separate from the AI-generated cache (smart_reporter_cache) and from admin-created content (verified or unverified). The database distinguishes them by the category field and the presence of user_id.

Editor Right Column: Adaptive Panels

The editor's right column shows contextual panels based on intent and user actions:

- **AI Assistant (always visible)** — 3-layer response: corrections, answer, insights. This is the primary interactive tool for users who pasted their report (Gap 2/3).
- **Tool Panel (conditional)** — shown for tool_request intent. Embeds calculators/scoring tools via iframe.
- **Reference Panel (conditional)** — shown for reference/protocol/anatomy intents. Preview cards with view-full links.
- **Quick Check** — grammar/spelling/completeness review with accept/reject.
- **PII Warning** — always active, flags patient-identifying information.

Part D: Shared Admin Frontend (Code Deduplication)

Extract identical JS from both admin templates into `static/js/admin-resource-helpers.js` (~300 lines) and a Jinja2 macro templates/partials/resource_fields.html. This slims down both admin templates from ~700–800 lines to ~250–280 lines each.

Gap 1 Admin Integration

The admin resource fields macro is extended to support the new resource types from Gap 1. The admin UI for generating IF calculators and reporting templates now includes:

- URL input with “Add Reference” button (existing, enhanced with validation)
- DB reference picker: searchable dropdown to select existing IF calculators, reporting templates, protocols, or TNM entries from the database
- PDF upload: file input that extracts text server-side and stores it as a resource reference

These resource fields are shared between admin and user-facing generation. When a user provides resources via the Smart Reporter input (e.g. “use this article: [URL]”), the intent classifier extracts them and passes them through the same pipeline.

Implementation Phases (Revised)

Phase 1: Generator Unification + Resource Pipeline (Gap 1)

Create `clinical_tool_generator.py` with resource-aware generation. Existing generators become thin wrappers. New utilities: `extract_pdf_text`, `fetch_db_references`, `fetch_tnm_data`.

Verify: Admin generates IF calc + RT template with and without resources. Outputs match or exceed previous quality. Resources are properly injected into prompts.

Phase 2: Shared Admin Frontend (Gap 1 admin UI)

Extract shared JS and Jinja2 macros. Extend resource fields with DB picker and PDF upload. Slim down admin templates.

Verify: All admin CRUD works. Resources (URLs, DB refs, PDFs) work in all generation modals.

Phase 3: Intent Router + Landing Redesign (Gaps 2 & 3 entry points)

Add `classify_intent()` with Haiku. Redesign landing page with single input + "Paste my report" button. Add route-intent endpoint.

Verify: Various inputs route correctly. "Paste my report" opens editor with empty report area. Quick picks work. Sessions/templates still accessible.

Phase 4: Smart Caching + Personal Templates (Gap 4)

Enhance cache logic with personal template tier. Add personal template save prompt in editor. Add "Your Templates" section on landing page.

Verify: Paste report → save prompt appears. Personal template saved and retrievable. Cache hierarchy respects all four tiers.

Phase 5: Walkthrough Abort + Flexible Exit (Gap 3)

Add persistent toolbar to walkthrough scene with Exit to Editor, Generate Blank Template, and Paste My Report options. Implement state preservation on abort.

Verify: Abort at step 3 of 7 → partial findings appear in editor. Generate blank template produces appropriate structure. Resume option works on return.

Phase 6: Adaptive Editor Panels + Anatomy

Add Tool Panel (iframe embed), enhanced Reference Panel (preview cards), Anatomy Panel. Enhance relevant-content endpoint. Add anatomy endpoint.

Verify: "Show me Li-RADS" → tool panel embeds calculator. Pasted report + AI Assistant query works smoothly. Anatomy panel shows content.

Files Summary (Revised)

Phase	Action	File	Notes
1	CREATE	clinical_tool_generator.py	~300 lines (incl. Gap 1 resource utils)
1	MODIFY	incidental_findings/generator.py	363→~60 lines (thin wrapper)
1	MODIFY	reporting_template_generator.py	361→~60 lines (thin wrapper)
1	MODIFY	incidental_findings/routes.py	+embed route, shared import
1	MODIFY	reporting_routes.py	+embed route, shared import
2	CREATE	static/js/admin-resource-helpers.js	~350 lines (incl. Gap 1 DB picker)
2	CREATE	templates/partials/resource_fields.html	~80 lines (incl. PDF upload)
2	MODIFY	templates/incidental_findings/admin.html	729→~250 lines
2	MODIFY	templates/admin_reporting_templates.html	805→~280 lines
3	MODIFY	ai_smart_reporter.py	+classify_intent (~60 lines)
3	MODIFY	reporting_routes.py	+route-intent endpoint
3	MODIFY	templates/smart_reporter.html	Landing redesign + paste entry
4	MODIFY	reporting_routes.py	+personal template CRUD, save prompt API
4	MODIFY	templates/smart_reporter.html	Your Templates section, save prompt UI
5	MODIFY	templates/smart_reporter.html	Walkthrough toolbar + abort paths
5	MODIFY	reporting_routes.py	+blank template generation endpoint
6	MODIFY	templates/smart_reporter.html	Tool, Reference, Anatomy panels
6	MODIFY	reporting_routes.py	+anatomy endpoint, enhanced relevant-content
6	MODIFY	ai_smart_reporter.py	+anatomy reference (~50 lines)

What Does NOT Change

- Database models (IncidentalFindingCalculator, ReportingTemplate, ClinicalProtocol) — untouched. Personal templates use existing ReportingTemplate with new category + user_id.
- All existing public URLs — no redirects needed
- Backup/restore format — untouched
- Standalone tool pages (IF finder, RT browse, TNM index, protocol view) — still accessible via global nav
- Existing walkthrough scene internals — still works, enhanced with abort toolbar
- Existing editor scene core — untouched, right panel enhanced
- PII Guard — untouched

Appendix: User Flow Diagrams

Flow A: Standard Walkthrough Path

User enters query → Intent classifier → Cache check → Serve cached or generate walkthrough → User completes walkthrough → Editor opens with generated report → AI Assistant available in side panel

Flow B: Skip to Editor (Gap 2)

User clicks “Paste my report” → Editor opens with empty report area → User pastes report → System detects paste and prompts “Save as personal template?” (Gap 4) → AI Assistant available in side panel for interactive help

Flow C: Abort Walkthrough (Gap 3)

User enters query → Walkthrough starts → User completes steps 1–3 of 7 → User clicks “Exit to Editor” → Editor opens with partial findings pre-populated → User can paste over or continue editing → AI Assistant available

Alternatively: User clicks “Generate Blank Template” → Quick Haiku call generates structured template → Editor opens with blank template → User fills in findings

Flow D: Resource-Aware Generation (Gap 1)

Admin or user provides resources (URLs, DB references, PDFs) alongside their query → Resources are extracted and formatted → Generator prompt includes resource context → Output is grounded in provided evidence → If no resources provided, AI uses own knowledge (current behaviour)